01 Recruiter essentials

Tell me about yourself.

My career has always been customer-facing, from retail and frontline support through IT and nearly seven years in GitLab Support Engineering. I enjoy the combination of deep technical investigation and helping the actual person affected by the problem. I have worked through GitLab's growth into a public company and want to bring that experience to my next challenge.

Why are you looking to leave GitLab?

GitLab has given me an enormous amount professionally, and I have valued watching it evolve. I am ready for a new challenge rather than a fresh start. I want to apply my experience at a fast-moving, developer-focused company where I can keep learning and make a meaningful contribution.

What is a weakness, and what are you doing about it?

I can sometimes spread my attention across too many investigative threads, especially when a system is complicated. I have become more deliberate about mapping the problem first, deciding what evidence matters next and involving colleagues with deeper expertise instead of disappearing down several rabbit holes.

What questions do you have for us?

How has AI-assisted database creation changed Support's ticket volume and technical mix? What challenges and opportunities do you see for the Support team over the next year? And, if the recruiter works at Supabase: what has their personal experience of its culture been?

Why Supabase specifically?

I have used Supabase in my own projects and genuinely like its approach: instead of spending weeks integrating backend infrastructure, I can focus on building the application. Its PostgreSQL foundation, open-source approach and position in AI-assisted development excite me. Its Kaizen mindset, autonomy and ownership also resemble the culture in which I have thrived.

What are your greatest strengths?

Customer communication: I can explain complex technical issues without forgetting the person on the other end. **Systematic troubleshooting:** I investigate across components and narrow problems using evidence. **Learning:** I am comfortable admitting what I do not know, finding the right expertise and improving as I go.

Why Support Engineering rather than a purely engineering role?

I love working through technical problems and I enjoy building things, but the human connection is what brings me back to Support. There is real satisfaction in investigating something difficult and going back to a customer with an answer they can act on.

What salary are you targeting?

Around **£80,000 base salary**, while remaining open to discussing the full package, including equity and benefits. If presented with a lower figure, I can consider the overall offer without agreeing to a new target on the spot.

02 Behavioural examples

What achievement are you particularly proud of?

When I stepped into GitLab's licensing and renewals team to help cover a staffing gap, the training module still described old systems and processes. I had to learn much of the role from earlier tickets and practical work. As I discovered gaps, I rewrote the onboarding material and updated related customer guidance. By the time more colleagues joined, they had a current, substantially more comprehensive starting point rather than the outdated material I had inherited.

Tell me about a time you took ownership without being asked.

At Mothership, a serious token-theft incident erupted late at night while most of our distributed European team was offline. I noticed a sudden wave of customer reports, signed back in, gathered evidence from tickets and the exchange activity, and contacted the CEO immediately. I worked with leadership and legal so we could issue accurate, coordinated customer messaging that same night. I could not solve the underlying theft, but I helped the company respond to customers much sooner.

Tell me about a disagreement with a colleague or another team.

An enterprise customer urgently wanted changes to billing and compliance-related account information. Our documented process restricted how those changes could be made. A colleague was focused on the customer's need for speed; I agreed that urgency mattered but would not bypass the rules myself. Rather than argue, I involved the relevant managers so we could find an authorised approach. The request was ultimately resolved within SLA while following the required process.

Describe feedback that changed the way you work.

Earlier in my GitLab career, I was investigating a complicated ticket and had followed an error message into several separate threads. A colleague reviewing the case told me I was focusing on the wrong area. I initially challenged that assessment because I wanted to understand his reasoning. We paired on the ticket, and he showed me that the error was a downstream symptom from a different component. It changed how I troubleshoot: map the system and test the likely origin before chasing each visible error.

03 Advocacy & problem-solving

Tell me about a time you advocated for a change outside your authority.

GitLab's licensing true-up system could generate unexpected charges, particularly painful for smaller customers. Working in Support, I repeatedly raised the need for clearer warnings and guardrails with people in a position to influence it. Controls eventually arrived, but later than I wanted. Looking back, I would have been more effective if I had gathered affected cases into a formal, evidence-backed proposal rather than relying mainly on conversations.

Tell me about a difficult technical problem you solved.

A customer ran GitLab's self-hosted AI Gateway in **offline mode**, where it should use model files already available on the machine. The gateway could not load a tokenizer (the component that prepares text for the model). I initially suspected the container image but checked that assumption and changed direction. The real issue was that the **HF_HOME** setting told the software to look in one cache folder, while the required files were stored in another. Removing the conflicting override let it find the files, and the customer confirmed tokenizer loading worked. I also opened a documentation MR about the pitfall.

Tell me about a time you made a solution useful beyond one customer.

A security-related GitLab service issue was affecting open-source users who did not have paid support. I connected those forum reports with a more detailed investigation carried out for a paying customer, then turned the troubleshooting and fix into documentation, a blog post and a community forum explanation. The aim was to make the solution accessible beyond the original ticket and reduce how often people had to rediscover it.

Tell me about a difficult customer and how you handled it.

Fabien

Can you describe a technically complex issue you resolved?

In a self-hosted GitLab AI case, the service was failing to authenticate, even though other symptoms were also appearing. Rather than treat all of those symptoms as one problem, I narrowed the core failure to missing authentication key material and guided the customer through checking and repairing it. The customer later confirmed that the central AI generation path was responding again. I treated remaining model or performance concerns as separate follow-up issues instead of claiming one fix had solved everything.

04 Technical stories with outcomes

Tell me about a time you helped uncover a product defect.

After a self-managed GitLab upgrade, a customer could no longer use Duo just before a planned workshop. At first, licensing or seat entitlement seemed plausible, so I checked those. Licensing was corrected, but the feature still failed. I connected the behaviour with other reports from the same upgrade line, gathered evidence, and raised the pattern with development. Engineering confirmed an upgrade regression and prepared a fix. After the patch, the customer reported core Duo functionality was working; a separate capability remained unavailable in that environment.

Tell me about a time you helped restore service under pressure.

During an upgrade-related incident, a customer could not access a GitLab configuration page. I joined a live troubleshooting session and used the available database evidence to identify a workaround that restored access to the page. That got the customer moving again while we kept the possibility of a broader underlying issue open. It illustrates the distinction between **restoring a service** and **proving the permanent root cause**.

Tell me about a time you explained a difficult technical issue clearly.

A customer's self-managed GitLab upgrade failed because their configuration still used an older self-signed-certificate setting that had been removed. I explained why the setting was no longer supported and how the new trusted-certificate approach worked. Rather than suggesting they turn off certificate checks, I guided them towards installing the certificate in the supported trusted-certs location and removing the obsolete option. The customer later confirmed that the staged upgrade completed successfully.

Tell me about a serious customer incident you helped manage.

A production customer reported more than 100 missing or inaccessible GitLab repositories. The initial check listed **111 missing entries**. I reviewed the errors, brought the appropriate people together and helped compare the affected repository paths with available snapshots. We organised recovery in controlled batches rather than restoring blindly. The customer later confirmed the restored repositories were accessible. We kept an outstanding wiki-data question separate and did not claim that the original cause of the incident had been identified.

05 Complex cases & lessons

Tell me about a complex problem you investigated.

A customer's GitLab Runner jobs were using Kubernetes and a cloud storage bucket for build caches. The puzzling part was that a **test job could access the bucket**, but Runner's cache upload and download still failed. I built my own test setup and reproduced the failure. That helped clarify an important distinction: the job runs inside one Kubernetes container, but the **Runner manager** may be responsible for creating the signed URL used for cache access. Successful permissions inside the job therefore did not prove the manager had the credentials it needed. We narrowed the question, but the customer grew frustrated by repeated checks and ultimately paused the case.

Tell me about a case you could not fully resolve.

A GitLab Pages customer was stuck in an authentication redirect loop. I reviewed the earlier investigations rather than asking them to repeat everything, and identified a possible route involving the Pages OAuth configuration and a known upstream behaviour. The customer also needed DNS or platform changes they could not make immediately. I documented what had been tested, what still needed verifying and the customer-side blocker. The ticket was eventually closed by agreement, **not because the redirect had been confirmed fixed**.

How do you approach an issue with several possible causes?

A self-managed GitLab installation had intermittent HTTP 500 errors across CI and repository operations. Instead of treating the 500 response as a single known cause, I requested logs from the times of failure and compared evidence across the application, Gitaly/repository storage and PostgreSQL. Database connection closures and possible repository-service version mismatch emerged as different leads. I narrowed subsequent information requests to the relevant periods and documented which checks should happen next. The record does not establish a single root cause or a final fix.

Tell me about a mistake you made and what you learned.

I handled a case where Duo appeared disabled or unresponsive in a customer's IDE. We spent time arranging and conducting a troubleshooting call, but I later realised I had not established the exact IDE-extension version early enough. That should have been one of the first compatibility checks. We continued gathering logs, updated the engineering report and the customer later recovered after updating or reinstalling the editor and extensions. My lesson was to verify the basic environment and versions before investing in expensive troubleshooting.